MODULE 7.5

Cloud Data Security

States of Data | Cryptography | Hashing

Presented by: Kudzaishe Majeza

Cisco Networking Academy | 2026

Topics:

- 7.5.1 States of Data
- 7.5.2 Cryptography
- 7.5.3 Hashing
- 7.5.4 Check Your Understanding: Cloud Data Security
- Summary
- References

States of Data

Data security controls depend on which of three states the data is currently in:

State	Description	Primary Protections
Data at Rest	Stored on disk — databases, file systems, backups, archives.	Encryption of storage volumes/files, strict access controls.
Data in Transit	Actively moving across a network, between a client and server or between data centers.	TLS/SSL, VPNs, and other secure transport protocols.
Data in Use	Actively being processed in memory or by the CPU.	The hardest state to protect — techniques include encrypted memory and confidential computing (secure enclaves).

The same piece of data moves through all three states during its lifetime — a password is at rest in a database, in transit during login, and in use the moment the application checks it.

Cryptography

- **Symmetric encryption:** the same key encrypts and decrypts (e.g., AES) — fast and efficient for bulk data, but both parties need a way to share that one key securely first.
- **Asymmetric encryption:** uses a mathematically linked public/private key pair (e.g., RSA) — solves the key-distribution problem, since the public key can be shared openly, but it's significantly slower than symmetric encryption.
- **How TLS combines both:** a TLS connection uses asymmetric cryptography just to securely exchange a one-time symmetric session key, then switches to fast symmetric encryption for the actual bulk data — getting the security benefit of asymmetric crypto with the speed of symmetric crypto.
- **Key management is everything:** the strength of any cryptosystem depends entirely on how well its keys are protected — a mathematically perfect algorithm with a leaked key provides zero real security.

Abbreviations: AES = Advanced Encryption Standard. RSA = Rivest–Shamir–Adleman. TLS = Transport Layer Security.

Hashing

- **What it is:** a one-way function that takes any input and produces a fixed-size output (a digest) — unlike encryption, a hash can never be reversed back into the original data.
- **Hashing vs. encryption:** the distinction that matters most — encryption is two-way (reversible with the correct key); hashing is one-way (irreversible by design).
- **Password storage:** applications should store only the hash of a password, never the plaintext — so that even if the database is stolen, the original passwords aren't directly exposed.
- **Salting:** adding random data to a password before hashing it so the same password produces a different hash for every user, defeating precomputed "rainbow table" attacks.

Algorithm	Status
MD5	Broken — collisions are easy to produce; must not be used for security.
SHA-1	Deprecated — practical collision attacks exist; being phased out.
SHA-256 / SHA-3	Current standard — no known practical attacks.

Check Your Understanding

Identify the concept each scenario describes:

Q: A database file sitting on a storage volume, protected by disk encryption.

A: Data at rest

Q: A login form's traffic protected by TLS as it crosses the internet.

A: Data in transit

Q: Two parties exchange a one-time session key using RSA, then switch to AES for the rest of the session.

A: Asymmetric crypto for key exchange, symmetric crypto for bulk data (how TLS works)

Q: A stored password can't be "decrypted" back to its original text, even by the database administrator.

A: Hashing — it's one-way by design

Q: Two users with the identical password "password123" end up with two completely different stored values.

A: Salting

Q: An algorithm is flagged as unsafe because attackers can now deliberately produce two inputs with the same output.

A: A broken hash algorithm (like MD5 or SHA-1)

Summary

- **7.5.1 —** data needs different protections depending on its state: encryption for data at rest, secure transport protocols like TLS for data in transit, and confidential computing techniques for the harder problem of data in use.
- **7.5.2 —** symmetric encryption is fast but has a key-distribution problem; asymmetric encryption solves that problem but is slow — real systems like TLS use both together, asymmetric to exchange a key, symmetric for the actual data.
- **7.5.3 —** hashing is one-way by design, which is exactly why it's the right tool for password storage and integrity checks — encryption would be the wrong tool, because it's meant to be reversed.
- **The throughline:** cryptography and hashing solve different problems — confidentiality versus integrity/verification — and picking the wrong one for the job is a common, serious mistake.

References

Cisco Networking Academy — Network Defense, Module 7.5: Cloud Data Security

https://www.netacad.com

NIST — Guide to Storage Encryption Technologies (SP 800-111)

https://csrc.nist.gov/publications/detail/sp/800-111/final

NIST — Secure Hash Standard (FIPS 180-4)

https://csrc.nist.gov/publications/detail/fips/180/4/final